ΜΑΘΗΜΑ 4 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

# Σύνολα και λεξικά

Μοναδικά στοιχεία και αναζήτηση με «κλειδί»

Θεωρία + 3 ασκήσεις + 3 προγράμματα με λύσεις  Διάρκεια: 4 ώρες (3η–6η)  22 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 26/10/2026

# Τι θα ξέρεις στο τέλος

1. Να χρησιμοποιείς **σύνολα** (set) για να κρατάς μόνο **μοναδικά** στοιχεία.
2. Να κάνεις **ένωση, τομή και διαφορά** συνόλων.
3. Να φτιάχνεις **λεξικά** (dict) και να βρίσκεις μια τιμή από το **κλειδί** της.
4. Να διατρέχεις λεξικό με **items()** και να **μετράς συχνότητες**.
5. Να μετατρέπεις μια δομή σε άλλη: λίστα, πλειάδα, σύνολο, λεξικό.
6. Να διαλέγεις **ποια δομή** ταιριάζει σε κάθε πρόβλημα.

### Πώς είναι οργανωμένο το 4ωρο

**Ώρες 1–2:** σύνολα και λεξικά με live coding. **Ώρες 3–4:** εργαστήριο σε ζευγάρια — αφαίρεση διπλοτύπων, συχνότητα λέξεων και στο τέλος ένας **τηλεφωνικός κατάλογος με μενού**.

**ΣΎΝΔΕΣΗ** Με τις λίστες και τις πλειάδες της προηγούμενης φοράς, σήμερα ολοκληρώνουμε τις τέσσερις βασικές δομές δεδομένων της Python.

# Θυμάσαι από την προηγούμενη φορά;

**ΕΚΦΏΝΗΣΗ**

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι δίνει κάθε έκφραση;

**ΛΎΣΗ**

| ΈΚΦΡΑΣΗ | ΑΠΟΤΈΛΕΣΜΑ | ΓΙΑΤΊ |
|---|---|---|
| `len([1, 2, 3])` | `3` | πλήθος στοιχείων |
| `[10, 20, 30][-1]` | `30` | το τελευταίο |
| `[1, 2] + [3]` | `[1, 2, 3]` | ένωση λιστών |
| `(3, 4)[0]` | `3` | πλειάδα: ίδιοι δείκτες |
| `[x * 2 for x in [1, 2, 3]]` | `[2, 4, 6]` | comprehension |

### Γιατί αυτά

Οι λίστες βρίσκουν στοιχεία με τη **θέση** τους. Σήμερα θα δούμε δομές που δουλεύουν **χωρίς θέσεις**: με μοναδικότητα (σύνολο) και με κλειδί (λεξικό).

**ΘΥΜΉΣΟΥ** Η πλειάδα (x, y) δεν αλλάζει. Θα τη χρειαστούμε ξανά σήμερα, ως κλειδί λεξικού και στο items().

# Δύο ερωτήσεις που η λίστα απαντά άβολα

«ΠΟΙΟ ΕΊΝΑΙ ΤΟ ΤΗΛΈΦΩΝΟ ΤΟΥ ΝΊΚΟΥ;» · ΜΕ ΛΊΣΤΕΣ

```
names = ["Άννα", "Νίκος"]
phones = ["6971111111", "6972222222"]

i = names.index("Νίκος")
print(phones[i])
```

ΜΕ ΛΕΞΙΚΌ

```
phone = {"Άννα": "6971111111",
         "Νίκος": "6972222222"}

print(phone["Νίκος"])
```

«ΠΌΣΟΙ ΔΙΑΦΟΡΕΤΙΚΟΊ ΕΠΙΣΚΈΠΤΕΣ;» · ΜΕ ΛΊΣΤΑ

```
visits = ["Άννα", "Νίκος", "Άννα"]
unique = []
for v in visits:
    if v not in unique:
        unique.append(v)
print(len(unique))
```

ΜΕ ΣΎΝΟΛΟ

```
visits = ["Άννα", "Νίκος", "Άννα"]
print(len(set(visits)))
2
```

**Η ΙΔΈΑ** Κάθε δομή είναι φτιαγμένη για ένα είδος ερώτησης. Η σωστή επιλογή κάνει τον κώδικα **μικρότερο και γρηγορότερο**.

# set: κάθε στοιχείο μία φορά

```
colors = {"κόκκινο", "πράσινο", "μπλε"}

nums = set([1, 2, 2, 3, 3, 3])
print(nums)
{1, 2, 3}

print(len(set("banana")))
3

empty = set()      # ΌΧΙ {}
```

### Η σύνταξη

**Άγκιστρα { }** με στοιχεία χωρισμένα με κόμμα — σαν τα σύνολα των μαθηματικών.

### Μοναδικά στοιχεία

Τα διπλότυπα **εξαφανίζονται αυτόματα**. Το set(λίστα) είναι ο πιο γρήγορος τρόπος να τα αφαιρέσεις.

### Χωρίς σειρά

Δεν έχει θέσεις: το **colors[0]** δίνει TypeError. Και όταν τυπώνεται, η σειρά μπορεί να είναι οποιαδήποτε.

### Η παγίδα του {}

Το **{}** σκέτο είναι **κενό λεξικό**, όχι σύνολο. Για κενό σύνολο γράφεις **set()**.

**ΠΟΎ ΧΡΗΣΙΜΕΎΕΙ** Μοναδικοί επισκέπτες, tags χωρίς επαναλήψεις, λέξεις που χρησιμοποιήθηκαν σε ένα κείμενο.

# Ένωση, τομή, διαφορά

```
python = {"Άννα", "Νίκος", "Ελένη"}     # έκαναν το μάθημα Python
java   = {"Νίκος", "Μαρία"}             # έκαναν το μάθημα Java
```

| ΠΡΆΞΗ | ΣΥΜΒΟΛΟ | ΑΠΟΤΈΛΕΣΜΑ | ΑΠΑΝΤΆ ΣΤΟ |
|---|---|---|---|
| ένωση | `python \| java` | Άννα, Νίκος, Ελένη, Μαρία | ποιοι έκαναν **έστω ένα**; |
| τομή | `python & java` | Νίκος | ποιοι έκαναν **και τα δύο**; |
| διαφορά | `python - java` | Άννα, Ελένη | ποιοι **μόνο** Python; |
| συμμετρική διαφορά | `python ^ java` | Άννα, Ελένη, Μαρία | ποιοι **μόνο ένα** από τα δύο; |

### Όπως στα μαθηματικά

Ένωση ∪, τομή ∩, διαφορά. Η Python απλώς χρησιμοποιεί τα σύμβολα **|**, **&**, **-**, **^** του πληκτρολογίου.

### Η σειρά μετράει μόνο στη διαφορά

**python - java ≠ java - python**. Το δεύτερο δίνει μόνο τη Μαρία.

**ΣΗΜΕΊΩΣΗ** Στην οθόνη η σειρά των ονομάτων μπορεί να βγει διαφορετική. Το σύνολο δεν κρατά σειρά — μετράει μόνο **ποια** στοιχεία έχει.

# Αλλαγές σε σύνολο

```
s = {1, 2}
s.add(3)
{1, 2, 3}
s.add(2)          # υπάρχει ήδη
{1, 2, 3}
s.discard(5)      # δεν υπάρχει: τίποτα
s.remove(5)
KeyError: 5
print(3 in s)
True
```

| ΜΈΘΟΔΟΣ | ΤΙ ΚΆΝΕΙ |
|---|---|
| `.add(x)` | προσθέτει (αν δεν υπάρχει ήδη) |
| `.remove(x)` | αφαιρεί · **σφάλμα** αν λείπει |
| `.discard(x)` | αφαιρεί · **σιωπηλά** αν λείπει |
| `x in s` | υπάρχει; |
| `len(s)` | πλήθος |

### Το in είναι αστραπή

Σε λίστα 1.000.000 στοιχείων το **in** ψάχνει ένα-ένα. Σε σύνολο βρίσκει την απάντηση **αμέσως**.

### Γιατί append δεν υπάρχει

Το «append» σημαίνει «στο τέλος». Ένα σύνολο δεν έχει τέλος, γι' αυτό η μέθοδος λέγεται **add**.

**ΠΌΤΕ DISCARD** Όταν δεν σε νοιάζει αν το στοιχείο υπήρχε. Αν πρέπει να υπάρχει και η απουσία του είναι λάθος, θέλεις **remove**.

# Πράξεις συνόλων

**ΕΚΦΏΝΗΣΗ**

Με τα δύο σύνολα δίπλα, βρες τι δίνει κάθε έκφραση. Πρώτα στο χαρτί, μετά στον υπολογιστή.

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
```

**ΛΎΣΗ**

| ΈΚΦΡΑΣΗ | ΑΠΟΤΈΛΕΣΜΑ | ΣΧΌΛΙΟ |
|---|---|---|
| `a \| b` | `{1, 2, 3, 4, 5}` | όλα, μία φορά |
| `a & b` | `{3, 4}` | κοινά |
| `a - b` | `{1, 2}` | μόνο στο a |
| `b - a` | `{5}` | μόνο στο b |
| `a ^ b` | `{1, 2, 5}` | όχι στα κοινά |
| `set([3, 1, 3])` | `{1, 3}` | χωρίς διπλότυπα |
| `len(set("Mississippi"))` | `4` | M, i, s, p |

### Έλεγχος

Το **a | b** πρέπει να έχει όσα στοιχεία το **a - b**, συν το **a & b**, συν το **b - a**: 2 + 2 + 1 = **5** ✓

**ΤΕΧΝΙΚΉ** Ζωγράφισε δύο κύκλους (διάγραμμα Venn) και γράψε κάθε αριθμό στη σωστή περιοχή. Οι απαντήσεις βγαίνουν με μια ματιά.

# dict: κλειδί → τιμή

```
phone = {"Άννα": "6971111111",
         "Νίκος": "6972222222"}

print(phone["Άννα"])
6971111111
print(phone["Μαρία"])
KeyError: 'Μαρία'
print(phone.get("Μαρία", "άγνωστο"))
άγνωστο
```

### Το κενό λεξικό

Γράφεται **{}** ή **dict()**. Γεμίζει σιγά-σιγά, όπως η λίστα με το append.

### Σαν ένα πραγματικό λεξικό

Ψάχνεις τη **λέξη** (κλειδί) και βρίσκεις τον **ορισμό** (τιμή). Δεν χρειάζεται να ξέρεις σε ποια θέση είναι.

### Η σύνταξη

**{ κλειδί: τιμή, κλειδί: τιμή }** — άγκιστρα, με άνω-κάτω τελεία ανάμεσα σε κλειδί και τιμή.

### [ ] ή get();

Το **[ ]** βγάζει **KeyError** αν λείπει το κλειδί. Το **get()** επιστρέφει None ή την τιμή που του δίνεις.

**ΣΤΟ ΔΙΑΔΊΚΤΥΟ** Σχεδόν όλα τα δεδομένα που στέλνουν οι ιστοσελίδες (JSON) είναι λεξικά. Θα τα συναντήσουμε ξανά στις εφαρμογές web.

# Προσθήκη, αλλαγή, διαγραφή

```
stock = {"μήλα": 10, "αχλάδια": 4}

stock["κεράσια"] = 25   # νέο κλειδί
stock["μήλα"] = 8       # αλλαγή τιμής
stock["αχλάδια"] += 6   # τώρα 10
del stock["κεράσια"]
n = stock.pop("μήλα")   # n = 8

print(stock, len(stock))
{'αχλάδια': 10} 1
```

| ΕΝΤΟΛΉ | ΤΙ ΚΆΝΕΙ |
|---|---|
| `d[k] = v` | **προσθέτει** ή **αλλάζει** |
| `del d[k]` | διαγράφει το κλειδί |
| `d.pop(k)` | διαγράφει και **επιστρέφει** |
| `k in d` | υπάρχει το **κλειδί**; |
| `len(d)` | πλήθος ζευγαριών |

### Το in ψάχνει κλειδιά

Το **"μήλα" in stock** ρωτάει για **κλειδί**. Για τιμή γράφεις **10 in stock.values()**.

### Ίδια εντολή, δύο ρόλοι

Το **d[k] = v** φτιάχνει το κλειδί αν δεν υπάρχει, αλλιώς **αντικαθιστά** την παλιά τιμή.

**ΠΡΟΣΟΧΉ** Το **stock["αχλάδια"] += 6** δουλεύει μόνο αν το κλειδί **υπάρχει ήδη**. Αλλιώς KeyError — θα δούμε τη λύση σε λίγο.

# for πάνω σε λεξικό

```
grades = {"Άννα": 17, "Νίκος": 9, "Ελένη": 15}
```

| ΓΡΆΦΕΙΣ | ΠΑΊΡΝΕΙΣ ΣΕ ΚΆΘΕ ΓΎΡΟ | ΠΑΡΆΔΕΙΓΜΑ |
|---|---|---|
| `for k in grades:` | τα **κλειδιά** | Άννα, Νίκος, Ελένη |
| `for v in grades.values():` | τις **τιμές** | 17, 9, 15 |
| `for k, v in grades.items():` | **ζευγάρια** (κλειδί, τιμή) | ("Άννα", 17), … |

```
for name, g in grades.items():
    print(f"{name}: {g}")

avg = sum(grades.values()) / len(grades)
print(f"Μ.Ο.: {avg:.2f}")
Άννα: 17 ... Μ.Ο.: 13.67
```

### Το items() δίνει πλειάδες

Και τις «ξεπακετάρουμε» σε **name, g** — όπως το unpacking της προηγούμενης φοράς.

### Κρατά σειρά

Το λεξικό θυμάται τη **σειρά εισαγωγής** των κλειδιών.

**ΤΑΞΙΝΌΜΗΣΗ** Το **for name in sorted(grades):** διατρέχει τα κλειδιά **αλφαβητικά**.

# Τι επιτρέπεται για κλειδί

| ΚΩΔΙΚΑΣ | ΑΠΟΤΈΛΕΣΜΑ | ΓΙΑΤΊ |
|---|---|---|
| `{"a": 1, "a": 2}` | `{'a': 2}` | κλειδιά **μοναδικά**: κερδίζει το τελευταίο |
| `{(37.98, 23.73): "Αθήνα"}` | εντάξει | πλειάδα **δεν αλλάζει** → μπορεί |
| `{[1, 2]: "x"}` | **TypeError** | λίστα **αλλάζει** → δεν μπορεί |
| `{"Άννα": [17, 15, 18]}` | εντάξει | η **τιμή** μπορεί να είναι οτιδήποτε |

### Κλειδιά: μόνο αμετάβλητα

Αριθμοί, κείμενα, πλειάδες. Αν το κλειδί άλλαζε, το λεξικό δεν θα μπορούσε να το ξαναβρεί.

### Τιμές: οτιδήποτε

Αριθμοί, κείμενα, λίστες, ακόμα και **άλλα λεξικά**. Έτσι χτίζονται σύνθετα δεδομένα.

**ΝΑ ΘΥΜΆΣΑΙ** Εδώ φαίνεται γιατί υπάρχει η πλειάδα: μια θέση (x, y) μπορεί να γίνει κλειδί, μια λίστα [x, y] όχι.

# Βήμα-βήμα: τι έχει το λεξικό;

**ΕΚΦΏΝΗΣΗ**

Γράψε πώς είναι το λεξικό **μετά από κάθε γραμμή**. Τι τυπώνει στο τέλος;

```
d = {"x": 1}
d["y"] = 2
d["x"] = 5
d["z"] = d["x"] + d["y"]
del d["y"]
print(len(d), d.get("y", 0))
```

### Οι δύο λεπτομέρειες

Το **d["x"] = 5** δεν φτιάχνει νέο κλειδί, αλλάζει το υπάρχον. Το **get("y", 0)** δίνει 0 γιατί το y διαγράφηκε.

**ΛΎΣΗ**

| ΓΡΑΜΜΉ | D |
|---|---|
| `d = {"x": 1}` | `{'x': 1}` |
| `d["y"] = 2` | `{'x': 1, 'y': 2}` |
| `d["x"] = 5` | `{'x': 5, 'y': 2}` |
| `d["z"] = ...` | `{'x': 5, 'y': 2, 'z': 7}` |
| `del d["y"]` | `{'x': 5, 'z': 7}` |

### Τυπώνει

**2 0**

**ΤΕΧΝΙΚΉ** Όπως με τις λίστες: μια γραμμή του πίνακα για κάθε εντολή. Το λεξικό το γράφεις ολόκληρο κάθε φορά.

# Μετράμε με λεξικό

```
text = "abracadabra"
counts = {}

for ch in text:
    counts[ch] = counts.get(ch, 0) + 1

print(counts)
```

```
{'a': 5, 'b': 2, 'r': 2,
 'c': 1, 'd': 1}
```

### Η μαγική γραμμή

**counts.get(ch, 0) + 1**: «πάρε πόσες φορές το έχω δει ως τώρα — **0 αν είναι η πρώτη** — και πρόσθεσε 1». Έτσι δεν βγαίνει ποτέ KeyError.

| ΓΡΆΜΜΑ | COUNTS ΠΡΙΝ | COUNTS ΜΕΤΆ |
|---|---|---|
| a | {} | {'a': 1} |
| b | {'a': 1} | {'a': 1, 'b': 1} |
| r | {'a': 1, 'b': 1} | {..., 'r': 1} |
| a | {'a': 1, ...} | {'a': 2, ...} |

### Το ίδιο με if

**if ch in counts:** counts[ch] += 1 **else:** counts[ch] = 1. Σωστό, αλλά μακρύτερο.

**ΠΑΝΤΟΎ** Ψήφοι, λέξεις, προϊόντα που πουλήθηκαν, επισκέψεις ανά σελίδα: όλα είναι «μέτρησε πόσες φορές» — ίδιο μοτίβο.

# Λεξικό με λίστες

```
grades = {"Άννα": [17, 15, 18],
          "Νίκος": [9, 11, 8]}

grades["Άννα"].append(20)

for name, gl in grades.items():
    avg = sum(gl) / len(gl)
    print(f"{name}: {avg:.2f}")
Άννα: 17.50   Νίκος: 9.33
```

### Πρόσβαση σε ένα στοιχείο

**grades["Νίκος"][0] → 9**: πρώτα το κλειδί του λεξικού, μετά ο δείκτης της λίστας.

### Δύο επίπεδα

Το **grades["Άννα"]** είναι μια **λίστα**. Οπότε της κάνεις ό,τι κάνεις σε λίστα: append, sum, len.

### Ποιο είναι ποιο

Κλειδί = **όνομα μαθητή**. Τιμή = **λίστα βαθμών** του.

### Έτσι μοιάζει το JSON

Λεξικά με λίστες με λεξικά: η μορφή των δεδομένων του web.

**ΠΡΑΚΤΙΚΉ** Αν ο μαθητής δεν υπάρχει ακόμα: **grades.setdefault("Μαρία", []).append(14)** φτιάχνει τη λίστα και προσθέτει.

# Από τη μία δομή στην άλλη

| ΓΡΆΦΕΙΣ | ΠΑΊΡΝΕΙΣ | ΤΎΠΟΣ |
|---|---|---|
| `set([1, 1, 2])` | `{1, 2}` | set |
| `sorted({3, 1, 2})` | `[1, 2, 3]` | list |
| `tuple([1, 2])` | `(1, 2)` | tuple |
| `list("abc")` | `['a', 'b', 'c']` | list |
| `dict(zip(["a", "b"], [1, 2]))` | `{'a': 1, 'b': 2}` | dict |
| `dict([("a", 1), ("b", 2)])` | `{'a': 1, 'b': 2}` | dict |
| `list(d.keys())` | `['a', 'b']` | list |
| `list(d.items())` | `[('a', 1), ('b', 2)]` | list |

### Ο κανόνας

Το όνομα του τύπου είναι και η συνάρτηση μετατροπής: **list()**, **tuple()**, **set()**, **dict()**.

### Λεξικό από δύο λίστες

Το **zip** ζευγαρώνει κλειδιά και τιμές, το **dict** τα κάνει λεξικό.

**ΓΙΑΤΊ SORTED ΚΙ ΌΧΙ LIST** Το **list(σύνολο)** δίνει τα στοιχεία με τυχαία σειρά. Το **sorted(σύνολο)** δίνει λίστα **ταξινομημένη**.

# Οι τέσσερις δομές μαζί

| | ΛΊΣΤΑ | ΠΛΕΙΆΔΑ | ΣΎΝΟΛΟ | ΛΕΞΙΚΌ |
|---|---|---|---|---|
| **Σύνταξη** | `[1, 2]` | `(1, 2)` | `{1, 2}` | `{"a": 1}` |
| **Σειρά** | ναι | ναι | όχι | ναι (εισαγωγής) |
| **Διπλότυπα** | ναι | ναι | όχι | όχι στα κλειδιά |
| **Αλλάζει;** | ναι | όχι | ναι | ναι |
| **Πρόσβαση** | `x[0]` | `x[0]` | μόνο in | `x["a"]` |
| **Τυπική χρήση** | σειρά στοιχείων | σταθερό «πακέτο» | μοναδικά | αναζήτηση με όνομα |

### Η ερώτηση που αποφασίζει

Με **θέση** τα ψάχνω; → λίστα / πλειάδα. Με **όνομα**; → λεξικό. Με μόνο «**υπάρχει ή όχι;**» → σύνολο.

**ΣΥΝΔΥΑΣΜΟΊ** Στην πράξη τις συνδυάζουμε: λίστα από πλειάδες, λεξικό με λίστες, λεξικό από σύνολα.

# Ποια δομή ταιριάζει;

**ΕΚΦΏΝΗΣΗ**

α) Τα διαφορετικά hashtags που χρησιμοποιήθηκαν σε 1.000 posts. β) Ο βαθμός κάθε μαθητή, βρίσκοντάς τον από το όνομα. γ) Η σειρά με την οποία μπήκαν πελάτες σε ουρά. δ) Μια ημερομηνία (ημέρα, μήνας, έτος). ε) Πόσες φορές εμφανίζεται κάθε λέξη σε ένα κείμενο.

**ΛΎΣΗ**

| | |
|---|---|
| α · σύνολο | Μας νοιάζουν μόνο τα **μοναδικά** — τα διπλότυπα φεύγουν μόνα τους. |
| β · λεξικό | Αναζήτηση με **όνομα**: όνομα → βαθμός. |
| γ · λίστα | Η **σειρά** μετράει και η ουρά αλλάζει (μπαίνουν, βγαίνουν). |
| δ · πλειάδα | Τρεις τιμές που ανήκουν **μαζί** και δεν αλλάζουν. |
| ε · λεξικό | Λέξη → πλήθος: το μοτίβο του **μετρητή**. |

**ΜΠΌΝΟΥΣ** Στο ε, πώς βρίσκεις πόσες **διαφορετικές** λέξεις έχει το κείμενο; → **len(counts)** ή **len(set(words))**.

# Αφαίρεση διπλοτύπων

**ΕΚΦΩΝΗΣΗ**

Μια λίστα με email εγγραφών έχει διπλές εγγραφές. Βρες πόσες είναι οι **μοναδικές** — και φτιάξε λίστα χωρίς διπλότυπα που **κρατά τη σειρά** εγγραφής.

**ΛΥΣΗ**

```
emails = ["a@x.gr", "b@y.gr", "a@x.gr",
          "c@z.gr", "b@y.gr"]

# 1. γρήγορα, χωρίς σειρά
unique = set(emails)
print(len(emails), "→", len(unique))

# 2. με τη σειρά εγγραφής
seen = set()
ordered = []
for e in emails:
    if e not in seen:
        seen.add(e)
        ordered.append(e)
```

```
5 → 3
['a@x.gr', 'b@y.gr',
 'c@z.gr']
```

### Γιατί seen είναι σύνολο

Γιατί το **in** σε σύνολο είναι **άμεσο** — η λίστα θα έψαχνε ένα-ένα.

### Ο λόγος για δύο λύσεις

Το set **χάνει τη σειρά**. Αν τη θες, χρειάζεται βρόχος.

**ΔΟΚΙΜΑΣΕ** Βάλε ένα email με κεφαλαία ("A@x.gr"). Μετράει ως διαφορετικό! Διόρθωση: **e.lower()** πριν τον έλεγχο.

# Συχνότητα λέξεων

**ΕΚΦΏΝΗΣΗ**

Ο χρήστης γράφει μια πρόταση. Εμφάνισε **πόσες φορές** εμφανίζεται κάθε λέξη (αλφαβητικά) και ποια είναι η **πιο συχνή**.

**ΛΎΣΗ**

```
text = input("Κείμενο: ").lower()
words = text.split()

counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

for w in sorted(counts):
    print(f"{w}: {counts[w]}")

best = ""
for w in counts:
    if best == "" or counts[w] > counts[best]:
        best = w
print("Πιο συχνή:", best, counts[best])
```

```
Κείμενο: Το παιδί και το σκυλί
  και το γατί
γατί: 1   και: 2
παιδί: 1  σκυλί: 1
το: 3
Πιο συχνή: το 3
```

### Δύο νέα εργαλεία κειμένου

**.lower()** κάνει όλα τα γράμματα πεζά, ώστε «Το» = «το». **.split()** σπάει το κείμενο σε λέξεις. Αναλυτικά στις συμβολοσειρές (23/11).

**ΕΠΈΚΤΑΣΗ** Πόσες **διαφορετικές** λέξεις έχει το κείμενο; Μία γραμμή: **len(counts)**.

# Τηλεφωνικός κατάλογος

```
phone = {}
while True:
    print("\n1.Προσθήκη 2.Αναζήτηση 3.Διαγραφή 4.Όλοι 0.Έξοδος")
    choice = input("Επιλογή: ")
    if choice == "1":
        name = input("Όνομα: ")
        phone[name] = input("Τηλέφωνο: ")
    elif choice == "2":
        name = input("Όνομα: ")
        print(phone.get(name, "Δεν βρέθηκε"))
    elif choice == "3":
        name = input("Όνομα: ")
        if name in phone:
            del phone[name]
            print("Διαγράφηκε")
        else:
            print("Δεν βρέθηκε")
    elif choice == "4":
        for name in sorted(phone):
            print(f"{name}: {phone[name]}")
    elif choice == "0":
        break
    else:
        print("Λάθος επιλογή")
```

**ΕΚΦΏΝΗΣΗ**

Κατάλογος με μενού: **προσθήκη**, **αναζήτηση**, **διαγραφή** και **εμφάνιση όλων** (αλφαβητικά). Τρέχει μέχρι να δοθεί 0.

### Ο σκελετός

Το **μενού με while True** του 2ου μαθήματος, με ένα **λεξικό** για αποθήκη.

### Το πρόβλημα που μένει

Μόλις κλείσει το πρόγραμμα, **οι επαφές χάνονται**. Την επόμενη φορά θα τις αποθηκεύουμε σε **αρχείο**.

**ΣΕ ΖΕΥΓΆΡΙΑ** Ο ένας γράφει τον κώδικα, ο άλλος τον δοκιμάζει με «κακές» εισόδους (όνομα που δεν υπάρχει, λάθος επιλογή). Μετά αλλάζετε ρόλους.

# Τι κρατάμε

### Σύνολο { }

**Μοναδικά** στοιχεία, χωρίς σειρά. **| & - ^** για ένωση, τομή, διαφορά. Κενό: **set()**.

### Λεξικό {k: v}

**Κλειδί → τιμή**. d[k] για πρόσβαση, **get()** για ασφαλή, **items()** για διάσχιση.

### Μετρητής

**counts[x] = counts.get(x, 0) + 1** — το πιο χρήσιμο μοτίβο της ημέρας.

Τώρα ξέρεις και τις **τέσσερις** βασικές δομές: **λίστα**, **πλειάδα**, **σύνολο**, **λεξικό**. Το μεγαλύτερο μέρος κάθε προγράμματος είναι η σωστή επιλογή ανάμεσά τους.

### Επόμενη φορά · Δευ 2/11

**Αρχεία και εξαιρέσεις**: ο κατάλογος θα αποθηκεύεται σε αρχείο, και τα λάθη του χρήστη δεν θα «ρίχνουν» πια το πρόγραμμα.

**ΓΙΑ ΤΟ ΣΠΊΤΙ** Διάβαζε ονόματα μέχρι κενή γραμμή. Εμφάνισε πόσα **διαφορετικά** δόθηκαν και **πόσες φορές** το καθένα.